F

TECHNICAL DOCUMENTATION

Finora Finance Tracker

Architecture, domain model, workflows, analytics, and security

Spring Boot 3.3.2 | Java 17 | 2026

System Overview

Finora is a server-rendered personal finance application for recording income and expenses, organizing transactions into categories, and analyzing the current month's financial position. Each account owns an isolated set of categories and transactions.

Version 2.0.0

This release introduces a complete editorial-ledger visual redesign with horizontal navigation, a warm paper-like canvas, cobalt actions, sharper metric panels, and responsive authentication, overview, transaction, and category screens.

Release: `https://github.com/arianaDaboc/finance-tracker/releases/tag/v2.0.0`
Walkthrough video: `https://github.com/arianaDaboc/finance-tracker/releases/download/v2.0.0/financial_tracker_video.mp4`

The browser receives complete HTML pages and PNG charts from one Spring Boot application. Business calculations, authorization, database access, page rendering, and chart generation are performed on the server. This design keeps the client lightweight and places financial rules inside the Java application.

Component	Technology	Responsibility
Runtime	Java 17 and Spring Boot 3.3.2	Application composition
Web	Spring MVC and Thymeleaf	Requests, forms, and HTML rendering
Security	Spring Security and BCrypt	Sessions, access control, password hashing
Persistence	Spring Data JPA and Hibernate	Entity mapping and queries
Database	PostgreSQL	Durable relational storage
Analytics	JFreeChart	Server-generated PNG charts

Architecture

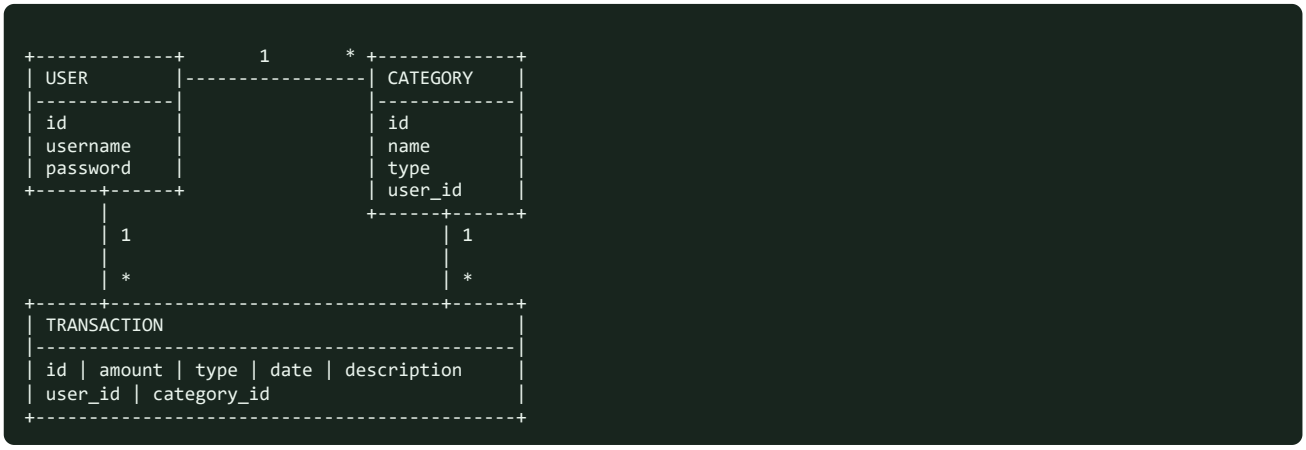
Finora follows a layered controller-service-repository architecture.



Controllers coordinate web use cases. They bind form data, obtain the authenticated identity, call services, and prepare Thymeleaf models. Services implement registration, ownership validation, aggregation, persistence coordination, and chart creation. Repositories provide entity operations and execute aggregate JPQL queries in PostgreSQL. Thymeleaf templates present the prepared model without owning business rules.

`AuthController` handles registration pages. `DashboardController` prepares monthly totals and recent activity. `TransactionController` and `CategoryController` coordinate record management. `ChartController` returns authenticated image responses.

Domain Model



`User` is both an authentication identity and the ownership boundary. Its password field stores a BCrypt hash rather than the submitted password. Username uniqueness prevents ambiguous identities.

`Category` classifies money as `INCOME` or `EXPENSE` and belongs to one user. Categories allow transaction totals to be grouped into meaningful financial areas such as salary, housing, food, or transport.

`Transaction` stores a positive `BigDecimal` amount, type, `LocalDate`, optional description, selected category, and owner. `BigDecimal` avoids the rounding errors associated with binary floating-point currency calculations. Transaction types are persisted as text, keeping database values independent of enumeration order.

Core Workflows

Identity and Session

```
Registration -> uniqueness check -> BCrypt hash -> user persistence
Sign in      -> user lookup -> hash comparison -> authenticated session
Request      -> session identity -> domain user -> protected operation
```

Spring Security delegates user lookup to `UserService`. After authentication, the identity is maintained in an HTTP session. Private controllers receive an `Authentication` object and resolve the corresponding domain user before accessing financial data.

Category and Transaction Processing

Category creation binds submitted fields to a new entity, assigns the authenticated user as owner, and persists it through `CategoryService`. Category lists are queried for that user only.

Transaction creation follows the same ownership pattern:

```
Form submission
      -> Spring MVC binds amount, type, category, date, description
      -> controller resolves authenticated user
      -> user is assigned as transaction owner
      -> service persists through TransactionRepository
      -> redirect reloads the transaction history
```

Records are listed in descending date order. Creation and deletion use `POST`, followed by a redirect. This redirect-after-post pattern prevents duplicate submissions when a page is refreshed.

For deletion, the service loads the requested record and compares its owner identifier with the authenticated user's identifier. A mismatch causes rejection. This second authorization layer protects against manually constructed requests, even though normal list queries already hide records belonging to other users.

Financial Processing

The dashboard derives an inclusive range from the current `YearMonth`: the first and last calendar day. PostgreSQL calculates aggregates for the authenticated user and date range.

```
monthly income  = SUM(amount WHERE type = INCOME)
monthly expense = SUM(amount WHERE type = EXPENSE)
balance         = monthly income - monthly expense
```

Aggregate queries use `COALESCE` so an empty result becomes zero rather than null. The transaction list supplies the activity count, while its first five date-ordered elements become the recent activity feed.

Category analytics are also calculated in PostgreSQL. Transactions are filtered by owner, type, and date range, grouped by category name, summed, and ordered by total amount. Database-side aggregation avoids loading an entire month into application memory solely to calculate totals.

Chart Pipeline

```
Chart request
   -> authenticated user and current month
   -> grouped repository query
   -> category name + aggregate amount
   -> JFreeChart dataset
   -> pie chart or vertical bar chart
   -> ChartUtils encodes PNG
   -> browser displays image
```

The expense chart uses a pie dataset to show each category's share of monthly spending. A placeholder dataset guarantees a valid image when no expenses exist. The income chart uses vertical bars to compare totals across income categories.

Charts are regular `image/png` responses. They require no browser chart library and remain behind the same authentication policy as financial pages.

Security Model

Security is applied at several levels:

- Spring Security permits only authentication pages and static styles without a session.
- BCrypt salts and hashes passwords before storage.
- CSRF protection applies to state-changing forms.
- repository methods scope lists and aggregates to the authenticated user;
- service methods independently verify ownership before deletion;
- JPA parameter binding avoids concatenating user values into query text;
- logout invalidates the active authentication context.

This defense-in-depth model separates authentication, query filtering, and operation authorization. Hiding a record in the interface is never treated as sufficient protection.

Interface and Extension Model

The version 2 interface uses an editorial ledger layout with horizontal navigation, a warm paper-like background, cobalt actions, serif display typography, financial metric panels, responsive forms, history tables, category lists, empty states, and separate authentication compositions. Green communicates income or positive state; coral communicates expense or destructive action. Responsive breakpoints reorganize navigation, grids, forms, tables, and authentication content for smaller screens.

The current scope is a personal ledger, not a complete accounting platform. It does not model bank accounts, transfers, recurring entries, budgets, shared households, multiple currencies, attachments, or audit history. The layered structure supports these extensions through new entities, repository queries, service rules, controller use cases, and matching views while preserving the central invariant: every financial operation belongs to and is authorized for one authenticated user.